Building Security Into Every Stage of the CI/CD Using DevSecOps

Revanth Kumar T S
Department of Computer Science and Engineering
Global Academy of Technology
Bengaluru, India
revanthkumarts123@gmail.com

Sanket
Department of Computer Science and Engineering
Global Academy of Technology
Bengaluru, India
sankethcoding@gmail.com

Vignesh P
Department of Computer Science and Engineering
Global Academy of Technology
Bengaluru, India
palanivignesh31@gmail.com

Shakthi Vel K
Department of Computer Science and Engineering
Global Academy of Technology
Bengaluru, India
shakthivelk1124@gmail.com

Sameena H S
Department of Computer Science and Engineering
Global Academy of Technology
Bengaluru, India
sameena.hs@gat.ac.in

***Abstract* — DevOps brought development and operations together, and release cycles improved as a result. Security did not get the same treatment. For most teams it stayed at the back of the process — reviewed late, often under time pressure, and long after the decisions that actually mattered had already been made. For a while that was manageable. It stopped being manageable as applications became more exposed and breaches more consequential.**

DevSecOps was the practical response to that shift. The core argument is straightforward: waiting until deployment to ask security questions means the answers come too late to change anything.

Putting that into practice is where most pipelines struggle. Tools get layered onto workflows that were never designed to carry them. Developers focused on shipping features are not always equipped to act on what a scanner reports — and when scanners flag dozens of issues indiscriminately, most warnings get tuned out entirely.

The pipeline described in this paper treats security as a continuous concern rather than a final checkpoint. Git and Jenkins manage source control and delivery. SonarQube examines code before it runs; OWASP ZAP tests behavior once it does. Trivy inspects container images. Docker and Kubernetes handle deployment, while Prometheus and Grafana keep runtime behavior visible throughout.

Keywords — DevSecOps, CI/CD pipeline security, vulnerability scanning, container security, automated security testing, runtime monitoring.

I. INTRODUCTION

CI/CD pipelines did not become standard practice overnight. Early software teams managed releases manually, and the problems that created were hard to ignore — broken builds, missed deadlines, and update cycles that dragged on far longer than anyone wanted. When agile began gaining ground, the expectation of frequent, working releases made manual delivery genuinely unworkable. Pipelines filled that gap. They absorbed the repetitive parts of the release process and made shipping software something teams could do regularly without it turning into an ordeal every single time.

Containers arrived around the same period and were dealing with a different but equally familiar frustration. The scenario most developers have lived through at least once — code that runs cleanly on a local machine and then breaks the moment it lands somewhere else — was exactly what containers were built to address. The environment travels with the application, so the gap between development and production stops being a source of constant surprises.

As both practices spread, security started showing its cracks. The standard approach for a long time was to run security checks close to release, treating it as a final gate rather than an ongoing concern. That was never ideal, but lower release frequency kept it from becoming critical. Once teams started shipping more often, the consequences of late security reviews became much harder to absorb. Finding a vulnerability after weeks of subsequent work have been built on top of it is a different problem entirely — fixing it can unravel decisions that were already considered settled. DevSecOps grew out of that reality, making the case that security cannot be bolted on at the end of a process it was never part of to begin with.

What made the shift toward DevSecOps difficult for many teams was not disagreement with the idea but the practical reality of changing pipelines that were already running in production. Reworking a live delivery process carries risk, and most teams were under pressure to keep shipping rather than pause and rebuild their workflows from scratch. Security tools also came with their own learning curve. A scanner producing a list of flagged issues is only useful if the person reading it understands what those issues mean and has a clear path to addressing them. Many teams found themselves with tools installed but not genuinely integrated — security outputs that sat alongside the pipeline rather than inside it, reviewed occasionally rather than consistently, and rarely connected to any real accountability for resolution.

Fig. 1 shows how the three parts of DevSecOps — Development, Security, and Operations — are connected. Development covers writing code, putting different parts of the system together, and managing releases. Operations handles the infrastructure side of things, like keeping services available and making sure the system can handle the load it receives. Security is responsible for scanning, identifying risks, and putting controls in place. What the figure shows is that these three areas do not work one after another. They

work simultaneously each one affects the others. That is the core idea — none of these three can be separated from the rest without the whole approach falling apart.

Fig. 1. DevSecOps

One of the main changes DevSecOps brings is automating security checks. Doing security reviews manually performs poorly in situations where a team is releasing software frequently. It takes too long, and outputs vary without clear reason from one review to the next. When checks are automated and built into the pipeline they run every time a build happens. They apply the same rules every time and the results come back quickly. If something is wrong the whoever built that particular module finds at a point when context is still fresh working on that part of the system, not days or weeks later.

The framework put forward in this paper uses a set of tools that each handle a different part of the security work. SonarQube looks at the source code before anything is deployed. It checks for problems in how the code is written — things like patterns that are known to cause vulnerabilities or implementation that fails to satisfy quality standards. This occurs at an earlier stage of pipeline before the application is even running anywhere.

Once the application is running, two other tools come into play. OWASP ZAP tests the application while it is live by sending requests to it and reviewing the responses. If something concerning the manner in which application responds suggests a vulnerability is present, OWASP ZAP picks that up. Trivy works differently — it looks at the container image and checks its components against a list of known vulnerabilities. These two tools cover different things. OWASP ZAP is focused on how the application behaves when it is running. Trivy is focused on what the container image contains. Using both means more is covered than either tool could manage on its own.

Prometheus and Grafana handle monitoring once everything is deployed. Prometheus collects data from the running system — things like how fast it is responding, how many errors are occurring, and how much of its resources it is using. Grafana takes that data and displays it in dashboards that are easy to read. If something starts going wrong the dashboards show it straight away. This matters because catching a problem early — while it is just starting — is much easier to deal with than finding out about it after it has already affected users.

The overall point of putting all of this together is that security stops being something that only happens at certain points and becomes something that is always running in the background. A problem found early in development is a small issue. The same problem found right before a release is a much bigger one. This framework is built around the idea that finding problems early is not just better practice — it actually saves time, reduces cost, and makes the whole development process less stressful.

II. LITERATURE SURVEY

One thing early DevSecOps research kept bringing up was that security was getting pushed to the side inside DevOps pipelines. It was not being ignored completely, but it was clearly not being treated as a priority either. Nikolov and Aleksieva-Petrova (2023) wanted to look at this more closely and chose Jenkins as their environment. They put Snyk and OWASP ZAP into the pipeline at various points. The idea was to check which stages were actually picking up security issues and which ones were missing them. Finding vulnerabilities before deployment was shown to be possible, which was a useful outcome. That said, container security was not part of what they looked at, and nothing was set up to monitor the system after it went live.

Marandi et al. (2023) took a different approach entirely. Rather than Jenkins they used GitHub Actions as the base for their pipeline. Snyk and StackHawk were the tools they worked with, and Docker was running alongside everything. They did not stop at just scanning either — dashboards were added and parts of the remediation process were automated, so when issues came up the team had less manual work to get through. Even with those additions though, scanning container images and watching the runtime environment were still not covered.

Around this same period some researchers started looking beyond the technical side of things. Nayanajith and Wickramarachchi (2024) found that the biggest obstacles teams were hitting during DevSecOps adoption were not really about which tools to pick or how to configure them. Skill gaps were a common issue, and so was pushback came from engineers who were not comfortable changing approaches that had gone unchanged for a long time. On top of that, different groups within the same organisation were often not communicating well with each other. Fixing the technical setup was the easier part of the problem — shifting how people actually worked and collaborated on a daily basis was where things got genuinely difficult.

Bautista Ramos and Yoo (2025) took a step back from looking at any one specific case. Their work addressed several different aspects of already published studies, going through them to build up a general picture of what the research in this area actually looked like. What kinds of vulnerabilities were showing up most often across the literature, and what were researchers actually proposing to deal with them — those were the questions they were trying to answer. what became particularly apparent through their review was that a significant portion of the research they looked at had gone unexamined until that point outside of

controlled settings. That is a real limitation because it leaves open the question of how any of it would hold up in a live environment where conditions are messier and less predictable.

Some studies published in recent years started putting Jenkins, Docker, and Kubernetes together within identical pipeline and adding Prometheus and Grafana on top. Security was still part of the focus but so was overall system performance. Having Prometheus collecting data and Grafana displaying it meant teams gained genuine visibility into system behavior with their systems after deployment rather than discovering an underlying issue only after users started reporting problems. It was further noted that Trivy helps to check container-related issues.

In DevSecOps, some studies follow the CAMS model. This mainly includes culture, automation, measurement and sharing.. These factors are seen as important when applying DevSecOps in real situations These aspects are seen as useful when trying to use DevSecOps in practical situations. These factors matter when applying DevSecOps in real environments. It demonstrated a measurable effect on an effect on a strong effect on successfully applying DevSecOps in real environments.

Research has also examined improving the monitoring phase using machine learning. A hybrid intrusion detection system is applied during continuous monitoring, which helps in detecting threats with good accuracy. Still, these methods remain limited to specific datasets and need more real-world validation.

From these studies, it is observed that DevSecOps is improving, but some gaps still remain. From these studies, it appears that many works focus only on specific parts such as scanning or monitoring, but do not cover the complete pipeline.

Looking across these studies, a pattern becomes clear — most of them tackle one or two pieces of the pipeline rather than treating it as a whole. This work takes a different angle by connecting those pieces into something more complete. Static testing, dynamic testing, container scanning through Trivy, and continuous monitoring via Prometheus and Grafana are all brought into a single pipeline rather than handled separately.

This leads to the necessity of merging multiple stages of the pipeline into one system.

This work tries to piece these things together in a way that security can be checked at every step along the way.

III. PRINCIPLES OF SECURE DEVOPS

Often, organizations face problems when trying to use DevSecOps in actual projects. One common issue is that security is usually taken care of at the final stage, which later creates extra work. Speed is something development teams are always chasing, but cutting corners on security in that pursuit tends to create bigger problems down the line. Balancing the two is not straightforward — push too fast and vulnerabilities slip through, slow down too much and the whole delivery process suffers.

Then there is the knowledge gap, which is its own separate headache. Many developers are not trained to think like security professionals, and security teams rarely have a deep enough feel for how development actually works day to day. That disconnect breeds isolation between the two sides, and when teams stop talking to each other properly, the cracks start showing up in the product itself.

In our work, we tried to avoid these problems by adding security checks directly into the pipeline so they run together with development steps.

A. Principles Used in Our Work

1) Security from Early Stage

Instead of adding security later, checking is done from the beginning. Code is checked during development itself so that issues is observable in early.

2) Automation in Pipeline

A large number of steps are automated. When code is pushed, the pipeline starts and runs the required steps like checking and building without much manual work.

3) Continuous Process

The system works in a continuous manner where the flow is not interrupted. Code changes move step by step without stopping, which helps in faster development.

4) Multiple levels of Checking

Security is checked in more than a single way. Checking is done in stages. First the code is checked, then the application, and later the running system is verified.

5) Container Checking

Before deployment, the container is also checked so that any major issue can be noticed at that stage itself.

6) Monitoring After Deployment

After deployment, the system is still observed regularly. This helps in identifying if any issue comes later.

7) Team Involvement

All stages are connected, so work is not limited to one team. This makes it a bit easier for teams to follow the process and work together without much difficulty confusion.

8) Managed Setup

Deployment is handled in a controlled way so that the system stays stable and is easier to manage afterwards.

Using this approach, the system becomes more secure and simpler to manage. Issues can be noticed earlier, and overall the process becomes easier to deal with.

IV. DEVSECOPS PIPELINE MODEL

The pipeline considered in this work is given in Fig. 1. In our case, development and security are handled together as the process moves step by step, instead of doing them separately. The idea is to check issues at different stages, not only at the end.

Fig. 2. Proposed DevSecOps pipeline model

The process starts when code is taken from the Git repositoryhen the code is verified against SonarQube to detect some elementary issues. Next, code analyses with SonarQube to catch some basic issues. The application is build on Node. js depending on the project.

Once the code clears those checks, a Docker image gets built from it. That image goes straight into Trivy, which combs through the container looking for any known weaknesses before anything moves further. After that, Kubernetes takes over and handles the actual deployment of the application.

OWASP ZAP comes into play once the application is actually up and running, putting it through tests that reflect real usage conditions. Some vulnerabilities simply do not show themselves until the application is handling live requests, so this stage tends to catch what earlier checks missed.Prometheus pulls in data from the running system on an ongoing basis, while Grafana takes that data and turns it into something the team can actually read at a glance rather than piecing together numbers manually.

The pipeline as a whole follows a logical order where one stage naturally leads into the next. Security does not wait to be invited in further along in the process — carrying role at each step along the way. Because of that, problems tend to appear while they are still small, which makes addressing them far less disruptive to the overall development work.

V. METHODOLOGY

The framework put together in this work connects all the major stages of a DevSecOps pipeline into one continuous flow. Rather than treating security as something that gets reviewed separately, it runs alongside the development work at every point.

The whole process starts when a developer pushes their code to the Git repository. That push is what triggers the pipeline to begin. Before anything else happens the code goes through an initial check where basic syntax problems and quality issues get caught early.

After that the code goes into the build stage. Once the build is done a Docker image gets created from it. That image does not just move straight to the next stage though — Trivy runs a scan on it first to check whether there are any known vulnerabilities hiding inside the container.

Once that is done Kubernetes takes over and deploys the application so it is actually running. That is when OWASP ZAP comes into the picture. Rather than looking at the code as plain text the way earlier tools do, OWASP ZAP tests the application while it is live and running. Problems that only show up during runtime are what this stage is mainly trying to catch.

The final piece is ongoing monitoring. Prometheus collects performance data from the live system while Grafana presents it so that it is easy to interpret. Together, they help to ensure that anything unusual gets noticed quickly rather than going undetected. A summary of all the tools used across these stages is provided in Table I.

Sl. No	Tool	Category	Purpose
1	Git	Version Control	Track code changes
2	Jenkins	CI/CD Tool	Automate build and deploy
3	Docker	Containerization	Package application
4	Kubernetes	Container Orchestration	Manage containers
5	SonarQube	SAST Tool	Detect code issues

Table I: Tools Used in DevSecOps Pipeline

In with this approach, all stages are connected, and security checking is done at multiple points. Picking up issues at an early stage means far less work piles up as the later portions of cycle.

VI. RESULTS AND DISCUSSION

This work went through each portion of the broader DevSecOps pipeline individually, starting from the point where code first enters the system and following it all the way into production. The pipeline ran without interruption across all these stages, which itself confirmed that the integration held together under real conditions.

Starting with code verification made a noticeable difference — problems that might otherwise traveled further down the line got caught early, leaving fewer things to deal with in later stages.Container images got scanned before anything was allowed to proceed, so whatever reached the deployment stage had already been checked and cleared — reducing the chances of something problematic slipping through that layer.

Running tests on the application after it was already up and taking requests brought out a set of problems that had stayed hidden through everything before it. Some problems only show up when the application is actually doing something, and this stage was where those got surfaced. It enabled monitoring to observe the system behaviour, track performance and notice problems during run-time.

We noted that fewer bugs remain at the end when verification is done at different loops. This leads to less rework later on, as issues will be fixed earlier.

Across most implementations, development alongside security in the same flow improved things. Doing this also made the system a lot more stable and manageable than implementing security as an independent step.

VII. COMPARISON WITH RELATED WORK

Looking at earlier studies, most of them zeroed in on one or two parts of the pipeline — some focused purely on static code analysis, others only tested after deployment was already done. Security ended up being handled in patches rather than as something that ran through the whole process consistently.

The pipeline in this work covers all three levels — code, container, and post-deployment — which means security gets a look-in at each meaningful point rather than just one. That shift alone changes how vulnerabilities get caught and dealt with.

Something else that stood out in prior work was the absence of any monitoring after deployment. A lot of implementations simply stopped once the application went live. This work keeps going beyond that point, with Prometheus and Grafana staying active so the system can be watched while it is actually in use.

Stringing these stages together rather than treating them in isolation is what makes the difference — fewer gaps, fewer surprises, and less room for something to quietly go wrong. A side-by-side comparison of existing and proposed approaches is laid out in Table II below.

Feature	Existing Work	Proposed Work
Code Checking	Available	Included
Dynamic Testing	Partial	Included
Container Scan	Not Included	Included
Monitoring	Limited	Included
Full Pipeline	Not Covered	Included

Table II: Comparison of Different Approaches

Based on the table, this indicates that the proposed work covers more stages compared to earlier approaches.

VIII.CONCLUSION

What this work puts forward is a pipeline where development and security stop being two separate conversations. The core idea was straightforward — run checks at multiple points along the way rather than saving everything for the end.

Catching things early has a practical payoff. Problems that get flagged while the code is still moving through early stages are far simpler to address than ones that only turn up after everything has been built and packaged.

Some vulnerabilities are stubborn that way — they stay out of sight until the application is containerized or already deployed and responding to real requests. Having checks built into those later stages too means those ones do not get a free pass.

Once the application is live, monitoring keeps the visibility going. The system does not go unwatched just because deployment is finished — Prometheus and Grafana stay on through actual usage.

The pipeline runs in a fixed order, each stage completing before the next one begins. That structure keeps things predictable and makes sure nothing gets skipped, which in the long run saves the team from having to backtrack and redo work that should have been caught earlier.

In future, this can be improved by adding more features and making the automation better so that the system works in a smoother way.

REFERENCES

[1] M. Marandi, A. Bertia, and S. Silas, “Implementing and automating security scanning in a DevSecOps CI/CD pipeline,” in *Proc. World Conf. Communication & Computing (WCONF)*, Raipur, India, Jul. 2023.

[2] M. Voggenreiter, F. Angermeir, F. Moyón, U. Schöpp, and P. Bonvin, “Automated security findings management: A case study in industrial DevOps,” in *Proc. 46th Int. Conf. Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, Apr. 2024.

[3] A. Nayanajith and R. Wickramarachchi, “Challenges affecting the successful adoption of DevOps practices: A systematic literature review,” in *Proc. 4th Int. Conf. Advanced Research in Computing (ICARC)*, 2024.

[4] N. Dinh *et al.*, “Enhancing smart contract security through DevSecOps: An adaptive approach for vulnerability detection,” *IEEE Access*, vol. 13, pp. 159454–159520, 2025.

[5] R. Ashtagi *et al.*, “Building resilient CI/CD pipelines: A DevOps security-first framework,” in *Proc. Int. Conf. Computational, Communication and Information Technology (ICCCIT)*, 2025.

[6] R. C. B. Ramos and S. G. Yoo, “Cybersecurity in DevOps environments: A systematic literature review,” *IEEE Access*, vol. 13, pp. 191959–191980, 2025.

[7] S. A. Ruk *et al.*, “Effort estimation in Agile and DevOps: A systematic literature review of stakeholder dissatisfaction,” *IEEE Access*, vol. 13, pp. 167925–167960, 2025.

[8] N. S, P. Manja, P. Mathur, and P. B. Honnavalli, “Extensive review of threat models for DevSecOps,” *IEEE Access*, vol. 13, pp. 45252–45274, 2025.

[9] M. A. Akbar and A. A. Alsanad, “Empirical investigation of key enablers for secure DevOps practices,” *IEEE Access*, vol. 13, pp. 43698–43710, 2025.

[10] J. Ababneh *et al.*, “Enhancing DevOps continuous monitoring phase: Hybrid intrusion detection and ensemble learning system,” *IEEE Access*, vol. 14, pp. 4733–4755, 2026.